LAPORAN PRAKTIKUM STRUKTUR DATA

“ADT”

DISUSUN OLEH:

MUHAMMAD FATHAN EDLIN

2511537001

DOSEN PENGAMPU:

Dr. WAHYUDI, S.T, M.T

ASISTEN PRAKTIKUM:

SHELY SUKMADIRA

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS 2025

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat, taufik, dan hidayah-Nya, penulis dapat menyelesaikan makalah ini dengan baik dan tepat waktu. Makalah ini disusun sebagai salah satu bentuk pemenuhan tugas pada mata kuliah Struktur Data dengan pembahasan mengenai *Abstract Data Type* (ADT).

Makalah ini disusun dengan tujuan untuk memberikan pemahaman yang lebih mendalam mengenai konsep ADT, yang merupakan salah satu dasar penting dalam ilmu komputer, khususnya dalam perancangan dan pengelolaan struktur data. Dalam makalah ini, penulis membahas pengertian ADT, karakteristik, kelebihan, serta penerapannya dalam pemrograman. Penulis berharap materi yang disajikan dapat membantu pembaca dalam memahami konsep ADT secara lebih sistematis dan terstruktur.

Dalam proses penyusunan makalah ini, penulis menghadapi berbagai kendala, baik dalam hal pengumpulan referensi maupun dalam memahami materi secara menyeluruh. Namun, berkat bantuan, bimbingan, serta dukungan dari berbagai pihak, makalah ini akhirnya dapat diselesaikan. Oleh karena itu, penulis mengucapkan terima kasih yang sebesar-besarnya kepada

1. Dosen pengampu mata kuliah Struktur Data yang telah memberikan arahan dan bimbingan.
2. Teman-teman yang telah membantu dalam berdiskusi dan bertukar pikiran.
3. Semua pihak yang tidak dapat disebutkan satu per satu yang telah memberikan dukungan dalam penyusunan makalah ini.

Penulis menyadari bahwa makalah ini masih jauh dari kata sempurna. Hal ini disebabkan oleh keterbatasan pengetahuan dan pengalaman yang dimiliki penulis. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang bersifat membangun demi perbaikan dan penyempurnaan di masa yang akan datang.

Akhir kata, penulis berharap semoga makalah ini dapat memberikan manfaat, menambah wawasan, serta menjadi referensi yang berguna bagi pembaca, khususnya dalam memahami konsep *Abstract Data Type* (ADT) dalam mata kuliah Struktur Data.

Padang, 1 April 2026

Muhammad Fathan Edlin

DAFTAR ISI

KATA PENGANTAR……..……

DAFTAR ISI…….....

BAB I…….....

PENDAHULUAN……

1.1 Latar Belakang…….....

1.2 Tujuan…….

1.3 Manfaat…….....

BAB II…….....

PEMBAHASAN …….

2.1 Pengertian Abstrak Type (ADT)……

2.2 Karateristik ADT ……..……..

2.3 Kelebihan ADT…..…….……...

2.4 Contoh Penerapan ADT…….……

2.5 Contoh Sederhana ADT dalam Java …….…….

2.6 Praktikum Pekan 1…….……

BAB III……..……...

KESIMPULAN DAN SARAN……

3.1 Kesimpulan…….

3.2 Saran…….……..

DAFTAR PUSTAKA……...

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi yang semakin pesat menuntut adanya pengolahan data yang efektif dan efisien. Dalam ilmu komputer, struktur data menjadi salah satu komponen penting yang berperan dalam mengatur dan menyimpan data agar dapat digunakan secara optimal. Salah satu konsep dasar dalam struktur data adalah *Abstract Data Type* (ADT).

ADT merupakan suatu konsep yang mendefinisikan tipe data berdasarkan perilaku (operasi-operasi yang dapat dilakukan) tanpa memperhatikan implementasi detailnya. Dengan adanya ADT, seorang programmer dapat lebih fokus pada fungsi dan kegunaan suatu struktur data tanpa harus memikirkan bagaimana data tersebut disimpan secara internal.

Pemahaman mengenai ADT sangat penting karena menjadi dasar dalam pengembangan berbagai struktur data seperti stack, queue, list, dan lainnya. Oleh karena itu, pembahasan mengenai ADT perlu dipelajari secara mendalam agar dapat membantu dalam pengembangan program yang lebih terstruktur dan efisien.

1.2 Tujuan

Tujuan dari penulisan makalah ini adalah:

1. Menjelaskan pengertian Abstract Data Type (ADT).
2. Mengetahui karakteristik ADT.
3. Memahami kelebihan penggunaan ADT.
4. Mengetahui contoh penerapan ADT dalam pemrograman.

1.3 Manfaat

Manfaat dari penulisan makalah ini adalah:

1. Menambah wawasan tentang konsep ADT.
2. Membantu memahami dasar-dasar struktur data.
3. Menjadi referensi bagi pembaca dalam mempelajari ADT.

BAB II

PEMBAHASAN

2.1 Pengertian Abstract Data Type (ADT)

Abstract Data Type (ADT) adalah suatu konsep dalam pemrograman yang mendefinisikan tipe data berdasarkan operasi-operasi yang dapat dilakukan terhadap data tersebut, tanpa menjelaskan bagaimana data tersebut diimplementasikan. Dengan kata lain, ADT hanya berfokus pada *apa yang dapat dilakukan*, bukan *bagaimana cara melakukannya*. Hal ini memungkinkan programmer untuk menggunakan suatu struktur data tanpa perlu mengetahui detail implementasinya.

2.2 Karakteristik ADT

Beberapa karakteristik dari ADT adalah sebagai berikut:

1. Abstraksi
   Menyembunyikan detail implementasi dan hanya menampilkan operasi yang tersedia.
2. Enkapsulasi
   Data dan operasi dibungkus dalam satu kesatuan.
3. Interface (Antarmuka)
   ADT menyediakan kumpulan operasi yang dapat digunakan oleh pengguna.
4. Independensi Implementasi
   Implementasi dapat berubah tanpa memengaruhi cara penggunaan ADT.

2.3 Kelebihan ADT

Penggunaan ADT memiliki beberapa kelebihan, antara lain:

1. Mempermudah pemrogram dalam memahami program.
2. Meningkatkan modularitas program.
3. Memudahkan pemeliharaan dan pengembangan sistem.
4. Mengurangi kesalahan karena detail implementasi disembunyikan.

2.4 Contoh Penerapan ADT

1. Stack (Tumpukan)

Stack adalah struktur data dengan prinsip *Last In First Out* (LIFO).
Operasi utama:

- Push (menambah data)
- Pop (menghapus data)
- Peek (melihat data teratas)

2. Queue (Antrian)

Queue adalah struktur data dengan prinsip *First In First Out* (FIFO).
Operasi utama:

- Enqueue (menambah data)
- Dequeue (menghapus data)

3. List (Daftar)

List merupakan kumpulan data yang tersusun secara berurutan.
Operasi:

- Menambah data
- Menghapus data
- Mengakses data

2.5 Contoh Sederhana ADT dalam Java

```java
package pekan1_2511537001;

public class asalaja {

    // Class Stack
    static class Stack {
        private int[] data;
        private int top;
        private int maxSize;

        public Stack(int size) {
            maxSize = size;
            data = new int[size];
            top = -1;
        }

        // Cek apakah penuh
        public boolean isFull() {
            return top == maxSize - 1;
        }

        // Cek apakah kosong
        public boolean isEmpty() {
            return top == -1;
        }

        // Push
        public void push(int nilai) {
            if (isFull()) {
                System.out.println("Stack penuh!");
            } else {
                data[++top] = nilai;
                System.out.println(nilai + " ditambahkan ke stack");
            }
        }
```

```java
        // Pop
        public int pop() {
            if (isEmpty()) {
                System.out.println("Stack kosong!");
                return -1;
            } else {
                return data[top--];
            }
        }

        // Peek
        public int peek() {
            if (isEmpty()) {
                System.out.println("Stack kosong!");
                return -1;
            } else {
                return data[top];
            }
        }
    }

    public static void main(String[] args) {

        Stack s = new Stack(5);

        s.push(10);
        s.push(20);
        s.push(30);

        System.out.println("Isi teratas: " + s.peek());

        System.out.println("Pop: " + s.pop());
        System.out.println("Pop: " + s.pop());

        System.out.println("Isi teratas sekarang: " + s.peek());
```

2.6 Praktikum Pekan 1

JAM

```java
package pekan1_2511537001;

public final class jam_2511537001 {
        private int hh; // 0..23
        private int mm; // 0..59
        private int ss; // 0..59

        //---Validator---
        public static boolean isValid(int h, int m, int s) {
            return (0 <= h && h <= 23) && (0 <= m && m <= 59) && (0 <= s && s <= 59);

        }

        //---Konstruktor/Kreator---
        public jam_2511537001(int h, int m, int s) {
            this.hh = h; this.mm = m; this.ss = s;
        }

        // --- Selektor---

        public int getHour() { return hh; }
        public int getMinute() { return mm; }
        public int getSeconds() { return ss; }

        //--- Mutator---

        public void setHour (int h) {this.hh = h; }
        public void setMinute (int m) {this.mm = m; }
        public void setSecond(int s) {this.ss = s; }

        // --- Konversi ---
        public int toSeconds() { return hh * 3600 + mm * 60 + ss; }
        public static jam_2511537001 fromSeconds(int total) {
            if (total < 0) throw new IllegalArgumentException("detik negatif");
            total %= 24 * 3600; // wrap 24 jam
```

```java
            int h = total / 3600; total %= 3600;
            int m = total / 60;
            int s = total % 60;
            return new jam_2511537001(h, m, s);
        }

        // --- Relasional ---
        public int compareTo(jam_2511537001 other) { return Integer.compare(this.toSeconds(), other.toSeconds()); }
        public boolean equals(Object o) {
            if (!(o instanceof jam_2511537001 j)) return false;
            return hh == j.hh && mm == j.mm && ss == j.ss;
        }
        public int hashCode() {
            return java.util.Objects.hash(hh, mm, ss);
        }

        // --- Aritmatika ---
        public jam_2511537001 plus(jam_2511537001 other) { return fromSeconds(this.toSeconds() + other.toSeconds()); }
        public jam_2511537001 minus(jam_2511537001 other) { return fromSeconds(Math.floorMod(this.toSeconds() - other.toSeconds(),
        public jam_2511537001 nextSecond() { return fromSeconds(this.toSeconds() + 1); }
        public jam_2511537001 nextNSeconds(int n) { return fromSeconds(this.toSeconds() + Math.max(0, n)); }
        public jam_2511537001 prevSecond() { return fromSeconds(Math.floorMod(this.toSeconds() - 1, 24*3600));  }
        public jam_2511537001 prevNSeconds(int n) { return fromSeconds(Math.floorMod(this.toSeconds() - Math.max(0, n), 24*3600));

        // --- Durasi (detik) ---
        public static int durasiDetik(jam_2511537001 jaw, jam_2511537001 jakh) { return jakh.toSeconds() - jaw.toSeconds(); }

        // overriding ke string
        public String toString() {
            return String.format("%02d:%02d:%02d", hh, mm, ss);
        }
    public static void main(String[] args) {
        // TODO Auto-generated method stub

    }
```

JAM DRIVER

```java
package pekan1_2511537001;

public class jamdriver_2511537001 {
    public static void main(String[] args) {

        jam_2511537001 a = new jam_2511537001(23, 59, 50);
        jam_2511537001 b = new jam_2511537001(0, 0, 5);

        System.out.println("a              = " + a);
        System.out.println("b              = " + b);
        System.out.println("a+b            = " + a.plus(b));
        System.out.println("next 20s       = " + a.nextNSeconds(20));
        System.out.println("durasi (a,b)   = " + jam_2511537001.durasiDetik(a, b));
        System.out.println("a.compareTo(b) = " + a.compareTo(b));
    }
}
```

JAM DRIVER 2

```java
package pekan1_2511537001;

import java.util.Scanner;

public class jamdriver2_2511537001 {

    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.println("=== Program Driver Objek Jam ===");

        // 1. Input Jam Pertama
        System.out.println("\n--- Input Jam 1 ---");
        jam_2511537001 j1 = buatJamDariInput(input);

        // 2. Input Jam Kedua
        System.out.println("\n--- Input Jam 2 ---");
        jam_2511537001 j2 = buatJamDariInput(input);

        // 3. Menampilkan Data
        System.out.println("\n--- Hasil Operasi ---");
        System.out.println("Jam 1 (String) : " + j1.toString());
        System.out.println("Jam 2 (String) : " + j2.toString());
        System.out.println("Jam 1 dalam detik : " + j1.toSeconds());
        System.out.println("Jam 2 dalam detik : " + j2.toSeconds());

        // 4. Operasi Relasional
        int perbandingan = j1.compareTo(j2);

        if (perbandingan > 0) {
            System.out.println("Status : Jam 1 lebih lambat (setelah) Jam 2");
        } else if (perbandingan < 0) {
            System.out.println("Status : Jam 1 lebih awal (sebelum) Jam 2");
        } else {
            System.out.println("Status : Jam 1 sama dengan Jam 2");
```

```java
        }

    // 5. Operasi Aritmatika
        System.out.println("Durasi (J1 ke J2) : "
            + jam_2511537001.durasiDetik(j1, j2) + " detik");

        jam_2511537001 jNext = j1.nextSecond();
        System.out.println("Jam 1 Detik Berikutnya: " + jNext);

        jam_2511537001 jPrev = j1.prevSecond();
        System.out.println("Jam 1 Detik Sebelumnya: " + jPrev);

        // 6. Operasi Penjumlahan Jam
        jam_2511537001 jHasilPlus = j1.plus(j2);
        System.out.println("Hasil J1 + J2 : " + jHasilPlus);

        input.close();
        System.out.println("\nProgram Selesai.");
    }
    private static jam_2511537001 buatJamDariInput(Scanner sc) {

        int h, m, s;

        while (true) {
            System.out.print("Masukkan Jam (0-23) : ");
            h = sc.nextInt();

            System.out.print("Masukkan Menit (0-59): ");
            m = sc.nextInt();

            System.out.print("Masukkan Detik (0-59) : ");
            s = sc.nextInt();

            // Validasi
            if (jam_2511537001.isValid(h, m, s)) {
```

```java
                return new jam_2511537001(h, m, s);
            }

            System.out.println("[Error] Input tidak valid! Silakan ulangi.\n");
        }
    }
}
```

BAB III

KESIMPULAN DAN SARAN

3.1 KESIMPULAN

Abstract Data Type (ADT) merupakan konsep penting dalam struktur data yang mendefinisikan tipe data berdasarkan operasi yang dapat dilakukan tanpa memperhatikan implementasi detailnya. ADT memiliki karakteristik seperti abstraksi, enkapsulasi, dan independensi implementasi yang membuatnya sangat berguna dalam pengembangan program. Penggunaan ADT memberikan banyak keuntungan, seperti mempermudah pemahaman program, meningkatkan modularitas, serta memudahkan pemeliharaan sistem. Contoh penerapan ADT dapat ditemukan pada struktur data seperti stack, queue, dan list.

3.2 SARAN

Penulis menyarankan agar pembaca mempelajari konsep ADT secara lebih mendalam serta mencoba mengimplementasikannya dalam berbagai bahasa pemrograman. Dengan demikian, pemahaman mengenai struktur data akan semakin baik dan dapat diterapkan dalam pembuatan program yang lebih kompleks.

DAFTAR PUSTAKA

[1]Kodedasar.com, “Pengertian ADT,”.[Daring]. Tersedia pada: https://www.slideshare.net/slideshow/pengertian-adtpptx/257832996 [Diakses: 19-Sep-2025]

TUGAS MINGGUAN STRUKTUR DATA

1. Buatlah sebuah program ADT mobil yang memiliki atribut: nama, tahun, cc, harga, dan merk.

   Contoh: Avanza, 2020, 1300, 150000000, Toyota
2. Buatlah method ADT mobil tersebut: tambah mobil, hapus mobil, selector, dan mutator.
3. Buatlah 2 kelas: 1 kelas ADT mobil dan 1 kelas driver. Setiap kelas di lengkapi dengan nim.

JAWAB

1. ADT CLASS MOBIL

```java
package pekan1_2511537001;

import java.util.ArrayList;

// ==============================
// NIM: 2511537001
// CLASS ADT MOBIL
// ==============================
class Mobil {
    private String nama;
    private int tahun;
    private int cc;
    private double harga;
    private String merk;

    // Constructor
    public Mobil(String nama, int tahun, int cc, double harga, String merk) {
        this.nama = nama;
        this.tahun = tahun;
        this.cc = cc;
        this.harga = harga;
        this.merk = merk;
    }

    // ====================
    // SELEKTOR (GETTER)
    // ====================
    public String getNama() {
        return nama;
    }

    public int getTahun() {
        return tahun;
    }
```

```java
    public int getCc() {
        return cc;
    }

    public double getHarga() {
        return harga;
    }

    public String getMerk() {
        return merk;
    }

    // ====================
    // MUTATOR (SETTER)
    // ====================
    public void setNama(String nama) {
        this.nama = nama;
    }

    public void setTahun(int tahun) {
        this.tahun = tahun;
    }

    public void setCc(int cc) {
        this.cc = cc;
    }

    public void setHarga(double harga) {
        this.harga = harga;
    }

    public void setMerk(String merk) {
        this.merk = merk;
    }
```

```java
    // Method tampil
    public void tampilkan() {
        System.out.println(nama + ", " + tahun + ", " + cc + ", " + harga + ", " + merk);
    }
}


// ==============================
// NIM: 2511537001
// CLASS DRIVER
// ==============================
public class adt_mobil_2511537001 {

    public static void main(String[] args) {

        ArrayList<Mobil> daftarMobil = new ArrayList<>();

        // ====================
        // TAMBAH MOBIL
        // ====================
        Mobil m1 = new Mobil("Avanza", 2020, 1300, 150000000, "Toyota");
        Mobil m2 = new Mobil("Brio", 2021, 1200, 170000000, "Honda");

        daftarMobil.add(m1);
        daftarMobil.add(m2);

        System.out.println("=== Data Mobil ===");
        for (Mobil m : daftarMobil) {
            m.tampilkan();
        }

        // ====================
        // HAPUS MOBIL
        // ====================
        daftarMobil.remove(0);
```

```java

        System.out.println("\n=== Setelah Hapus ===");
        for (Mobil m : daftarMobil) {
            m.tampilkan();
        }

        // ======================
        // MUTATOR (UPDATE DATA)
        // ======================
        m2.setHarga(165000000);

        System.out.println("\n=== Setelah Update ===");
        m2.tampilkan();

        // ======================
        // SELEKTOR (AMBIL DATA)
        // ======================
        System.out.println("\nMerk Mobil: " + m2.getMerk());
    }
}
```

2. ADT CLASS DRIVER

```java
package pekan1_2511537001;

import java.util.ArrayList;

// ==============================
// NIM: 2511537001
// CLASS DRIVER
// ==============================
public class adt_driver_2511537001 {
    public static void main(String[] args) {

        ArrayList<Mobil> daftarMobil = new ArrayList<>();

        // TAMBAH MOBIL
        Mobil m1 = new Mobil("Avanza", 2020, 1300, 150000000, "Toyota");
        Mobil m2 = new Mobil("Brio", 2021, 1200, 170000000, "Honda");

        daftarMobil.add(m1);
        daftarMobil.add(m2);

        System.out.println("=== Data Mobil ===");
        for (Mobil m : daftarMobil) {
            m.tampilkan();
        }

        // HAPUS MOBIL
        daftarMobil.remove(0);

        System.out.println("\n=== Setelah Hapus ===");
        for (Mobil m : daftarMobil) {
            m.tampilkan();
        }

        // MUTATOR
        m2.setHarga(165000000);

        System.out.println("\n=== Setelah Update Harga ===");
        m2.tampilkan();

        // SELEKTOR
        System.out.println("\nMerk Mobil: " + m2.getMerk());
    }
}
```

3. OUTPUT FROM CLASS MOBIL DAN CLASS DRIVER

```
=== Data Mobil ===
Avanza, 2020, 1300, 1.5E8, Toyota
Brio, 2021, 1200, 1.7E8, Honda

=== Setelah Hapus ===
Brio, 2021, 1200, 1.7E8, Honda

=== Setelah Update ===
Brio, 2021, 1200, 1.65E8, Honda

Merk Mobil: Honda
```